Memory-Pooling Policy for Octopus

Pulak Mehrotra

Meetings With Yuhong/Richard

Objective

- **Designing demand-driven memory pooling policies for CXL-based cloud systems, targeting the Acadia/Octopus sparse topologies.**
- Main Questions:
  - How should memory allocation policies change when pooled memory is reachable only through sparse, island-based CXL connectivity?
  - How can the system distribute memory demand from “hot” hosts across available CXL devices without degrading latency (and eventually QoS)?
  - What tradeoffs emerge when designing dynamic memory pooling policies for sparse CXL topologies?
  - Can learning/RL-based approaches outperform heuristic policies in this setting?

FleetIO – An Overview

An example of RL being used for memory (SSD) allocation/pooling.

- *What is the problem?*
  - How should a cloud provider dynamically allocate limited SSD capacity across servers to maximize overall performance and efficiency under changing workloads?
- *Why is the problem hard?*
  - a) non-stationary + partially observable workloads, b) snowball effect of allocation, and c) large decision space
- *What is a general solution? Why can we not use it?*
  - *General Solution:* Model performance as a function of SSD allocation and formulate SSD allocation as a global optimization problem.
  - *Baselines:* static partitioning, equal-share allocation, and demand-based heuristic SSD resizing
  - Impractical in practice…
- *What is the suggested solution?*
  - FleetIO proposes a reinforcement-learning–based policy that learns allocation decisions directly from runtime telemetry, enabling adaptive, system-wide SSD allocation without explicit performance modeling.

RL Policy in FleetIO

The RL agent operates above the SSD virtualization layer, with admission control and block harvesting acting as safety and enforcement mechanisms.

- *Decentralized RL:* one agent per vSSD for scalable control
- *Implicit Coordination:* agent-aware reward balances local benefit and global contention
- *Coarse Decision-Making:* 2s
- RL model is trained offline and deployed.
- What is controlled in the hardware?

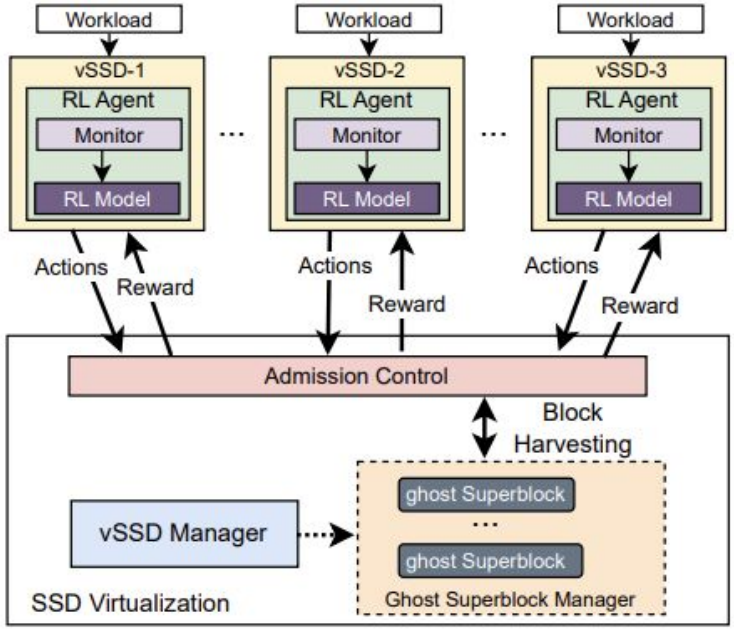


Figure 5. System architecture of FleetIO.

Things The Authors Did Well

The states and actions are well defined. Excellent modelling of the environment and pain points.

Table 1. Definition of RL states in FleetIO.

Symbol	Definition
Avg_BW_t	Average I/O bandwidth of the vSSD (MB/s)
Avg_IOPS_t	Average IOPS of the workload ($reqs/s$)
Avg_Lat_t	Average request latency of the vSSD (us)
SLO_Vio_t	Percentage of vSSD SLO violations (%)
$QDelay_t$	I/O request delay in a vSSD
RW_Ratio_t	Read/write ratio of the workload (%)
$Avail\_Capacity_t$	Available storage capacity (GB)
In_GC_t	Whether the vSSD is performing GC
$Cur\_Priority_t$	The current I/O request priority

Table 2. RL actions defined in FleetIO.

Symbol	Definition
$Harvest(gsb\_bw)$	Storage bandwidth to harvest
$Make\_Harvestable(gsb\_bw)$	Storage bandwidth harvestable for other vSSDs
$Set\_Priority(level)$	Set the I/O request priority

Reward

$$R_{single} = (1-\alpha) * \frac{Avg\_BW_{RL}}{Avg\_BW_{guar}} - \alpha * \frac{SLO\_Vio_{RL}}{SLO\_Vio_{guar}} \quad (1)$$

$$R_{i,\ multi\_agent} = \beta * R_{i,\ single} + (1-\beta) * \frac{\sum_{v \in V}^{v \neq i} R_{v,\ single}}{|V| - 1} \quad (2)$$

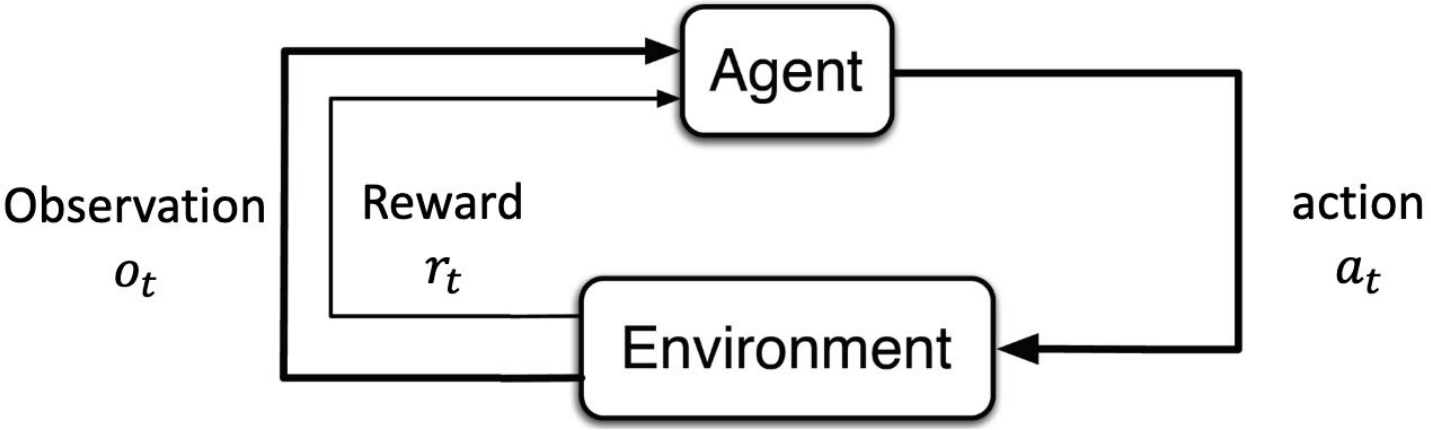
Objective

$$\max_{\pi} \mathbb{E}\left[\sum_{t=0}^{\infty} \lambda^t r_t\right]$$

Critiques

- *Why was the specific RL model was chosen?*
  - Why IPPO? There are far more mature multi-agent RL (eg. MARL) algorithms present in the field.
  - The reward function is slightly simplistic, and is not taking multi-agent communication and joint optimisation into context.
- *The decision-time frame seems too high for cloud workloads (microsecond spikes)*
  - Are the results accurate or do they smooth over longer windows?
- Multi-agent coordination could have problems scaling
  - What if there are 100+ vSSDs (realistic scale) instead of the 8 demonstrated by the paper?
  - Greedy neighbour problem! The policy is biased to certain workloads.
- The RL model is trained offline, relying on a simulator
  - There is the classical issue of is the simulator of sufficient fidelity compared to the real-life system
  - How are α and β (usually learnable parameters) selected for each workload? How does this handle workload non-stationarity?

RL In A Nutshell



Reinforcement learning (RL) is a machine learning approach where an agent learns to make optimal decisions by *trial and error*, receiving rewards or penalties to *maximize cumulative success*.

RL Policy – Environment Building

- States
  - What matters? What do we need to measure to decide what should go where?
- Actions
  - How do we control the available resources?
- Reward
  - What do maximise for? What do we avoid?
- Decision Making Timeframe
  - Per VM Arrival
- Model Choice/Behaviour
  - Multi-Agent vs Single Controller?

States

#	Category	Signal	Temporal Treatment	Name	Encoding
1	Resource Health	Per-MPD capacity utilization (% full)	3 windows, 1-hr	`mpd_cap`	$[v_t,\ \Delta_{1h},\ \Delta_{2h}] \times N_{\text{MPD}}$
2	Resource Health	Per-MPD bandwidth utilization (% full)	Current snapshot	`mpd_bw`	$[v_t] \times N_{\text{MPD}}$
3	Resource Health	Server memory pressure (% RAM utilized)	3 windows, 1-hr	`mem_press`	$[v_t,\ \Delta_{1h},\ \Delta_{2h}] \times N_{\text{servers}}$
4	Workload	Incoming VM size (requested memory)	Current snapshot	`vm_size`	Scalar (GB)
5	Workload	Predicted VM lifetime	Current snapshot	`vm_lifetime`	Scalar (hours)
6	Workload	Customer peak-to-average ratio	24-hr historical summary	`pta`	$\max(\text{usage}_{24h})$ / $\text{mean}(\text{usage}_{24h})$
7	Workload	Predicted future memory (growth rate)	Current snapshot	`future_mem`	Scalar (GB/hr)
8	Resource Health	Recent SLO violations (over-capacity events)	24-hr historical summary	`slo_vio`	Scalar (% allocations causing MPD overflow)
9	Topology	Server-to-MPD connectivity mask	Static	`conn_mask`	Binary vector $\in \{0,1\}^{N_{\text{MPD}}}$

Table 1: State space for the single central RL controller. v_t denotes the current value; Δ_{1h}, Δ_{2h} denote deltas from 1 and 2 hours prior respectively. `conn_mask` is a static topology signal used as a validity mask over the action space.

Objectives

Objective	**Corresponding State Signal(s)**	**State #**
Maximize resource utilization	Per-MPD capacity utilization	1
Minimize SLO violations	Recent SLO violations (over-capacity events); Per-MPD bandwidth utilization	4, 2
Minimize migration	Implicit — tracked externally as migration rate $\leq \tau_{\text{mig}}$	—

Table 1: Mapping between objectives and their corresponding state signals. Migration rate is not directly encoded as a state but is tracked externally as an evaluation metric.

Actions

At each VM arrival event, the agent outputs a continuous weight vector over the k MPDs reachable from the arriving VM's server:

$$\mathbf{a} = [w_1, w_2, \ldots, w_k] \in R^k \tag{1}$$

subject to the following constraints:

$$w_i \geq 0 \quad \forall\, i \tag{2}$$

$$\sum_{i=1}^{k} w_i = 1 \tag{3}$$

$$w_i = 0 \quad \text{if } \texttt{conn\_mask}_i = 0 \tag{4}$$

The weight vector is translated into a physical allocation as follows:

$$\text{GB allocated from MPD}_i = w_i \times \texttt{vm\_size} \tag{5}$$

The connectivity mask (`conn_mask`) is applied prior to the softmax output layer, zeroing out unreachable MPDs. The softmax then normalises the remaining weights to satisfy Equation 3.

#	**Action**	**Type**	**Abstraction Level**
1	Allocation split $\mathbf{a} = [w_1, \ldots, w_k]$ across connected MPDs	Continuous	High-level strategic → RL
2	`flag_migration(vm_id)` — flag a VM for migration	Binary	High-level strategic → RL
—	Physical page placement	Mechanical	Low-level → system
—	Address mapping updates	Mechanical	Low-level → system

Table 1: Action space for the single central RL controller. Actions 1 and 2 are high-level strategic decisions made by the RL agent. Physical page placement and address mapping are handled mechanically by the system and are transparent to the agent.

Reward

Option A: Fully Immediate, Smooth

All components are evaluated at VM arrival time. The SLO violation penalty is proportional and smooth, providing stable gradient signal for SAC.

$$R = \alpha \cdot \left(1 - \frac{\max(\texttt{mpd\_cap})}{\texttt{mpd\_cap\_max}}\right) - \beta \cdot \texttt{slo\_vio} - \gamma \cdot 1[\texttt{flag\_migration}] \tag{1}$$

where:

- $\max(\texttt{mpd\_cap})/\texttt{mpd\_cap\_max}$ is the peak MPD utilization normalized to capacity
- $\texttt{slo\_vio}$ is the proportion of recent allocations that caused MPD overflow
- $1[\texttt{flag\_migration}]$ is 1 if the agent flagged a VM for migration, 0 otherwise
- $\alpha,\ \beta,\ \gamma > 0$ are weights reflecting objective priority, with $\alpha < \beta$ to enforce SLO priority over utilization

Option B: Immediate + Delayed

Option B decomposes the reward into an immediate component at VM arrival and a delayed component at VM termination, discounted by the VM lifetime T. The immediate component provides smooth gradient signal; the delayed component applies hard penalties for actual SLO violations and migrations over the VM's lifetime.

$$R_t = \underbrace{\alpha \cdot \left(1 - \frac{\max(\texttt{mpd\_cap})}{\texttt{mpd\_cap\_max}}\right)}_{\text{immediate at arrival}} + \underbrace{\lambda^T \left(-\beta \cdot \texttt{slo\_vio}_T - \delta \cdot 1[\text{migrated}]\right)}_{\text{delayed at termination}} \tag{2}$$

where:

- T is the VM lifetime (hours), known at arrival via `vm_lifetime`
- $\lambda \in (0, 1)$ is the discount factor applied over the VM lifetime
- $\texttt{slo\_vio}_T$ is the proportion of time the VM experienced SLO violations during its lifetime
- $1[\text{migrated}]$ is 1 if the VM was migrated at any point during its lifetime
- $\beta \cdot \texttt{slo\_vio}_T$ and $\delta \cdot 1[\text{migrated}]$ are hard penalties reflecting objective priorities 2 and 3

Note: In both formulations, weights α, β, γ, δ are initially manually tuned and reflect the objective priority ordering (utilization $\prec$ SLO $\prec$ migration). Future work will explore online weight adaptation via multi-objective RL (MORL) to remove the need for manual tuning, directly addressing a known limitation of FleetIO [**?**].

Model Choice

First with no migration, and then with migration.

- *Primary Algo:* Soft-Actor Critic (SAC)
  - Due to our continuous state space, this would be the best fit
- *Fallback Algo:* Proximal Policy Optimization (PPO)
  - Historically, SAC has issue with temporal data. Let's compare with PPO just in case. We can fall back on TD3 too.
- *Baselines:* Greedy and Optimal

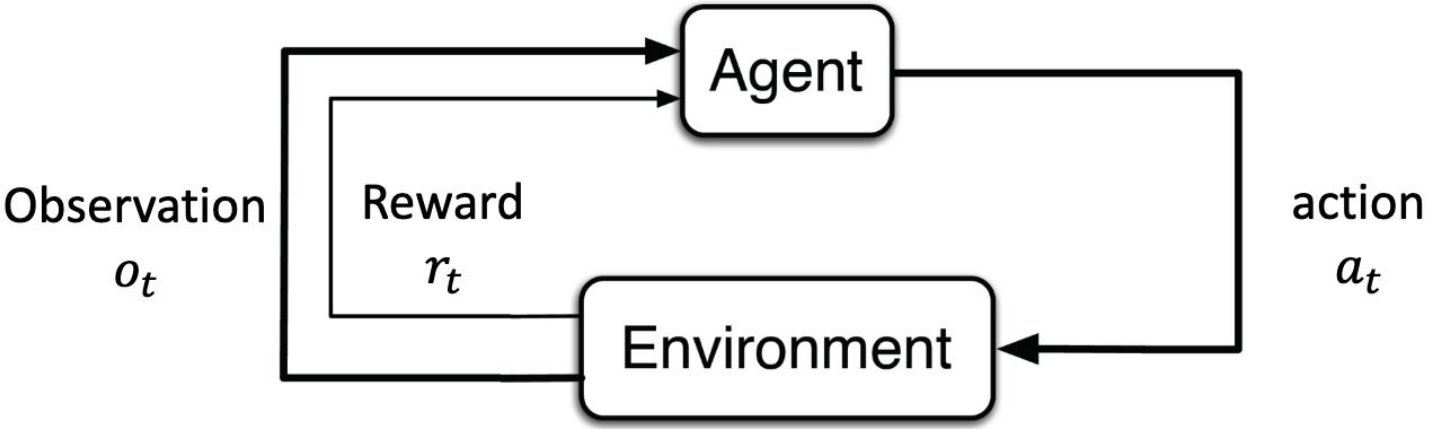
Will check this out this in the simulator we have!

Meetings With The Whole Team

Objective

- **Designing demand-driven memory pooling policies for CXL-based cloud systems, targeting the Acadia/Octopus sparse topologies.**
- Main Questions:
  - How should memory allocation policies change when pooled memory is reachable only through sparse, island-based CXL connectivity?
    - Compared to Pond (ASPLOS ‘23)
  - Can we make decisions based on minimising the need for migration in the future, while also maximising resource usage at the present instant?
  - Can learning/RL-based approaches outperform heuristic/control-theoretic policies in this setting?

RL In A Nutshell

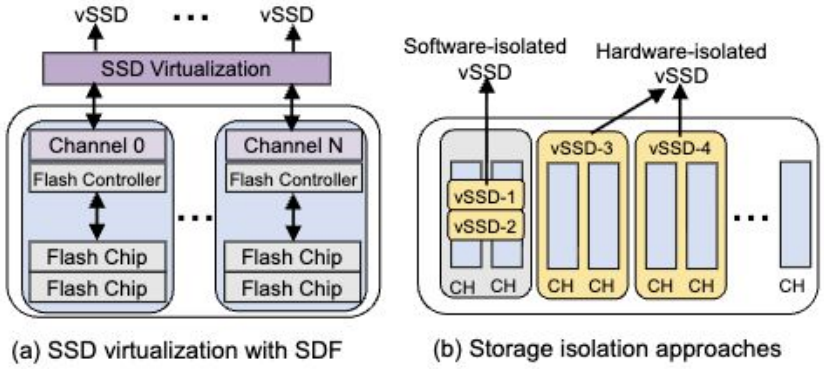


Reinforcement learning (RL) is a machine learning approach where an agent learns to make optimal decisions by *trial and error*, receiving rewards or penalties to *maximize cumulative success*.

RL In Computer Systems

Varying Implementations:

- Pensieve (SIGCOMM ‘17) - RL for Adaptive Bitrate Streaming
- Decima (SIGCOMM ‘19) - RL for Data Scheduling in Spark Workloads
- FleetIO (ASPLOS ‘25) - RL for SSD Allocation in the Cloud



FleetIO (ASPLOS ‘25) – An Existing RL Policy For Memory Management

- *What is the problem?*
  - How should a cloud provider dynamically allocate limited SSD capacity across servers to maximize overall performance and efficiency under changing workloads?
- *Why is the problem hard?*
  - a) non-stationary + partially observable workloads, b) snowball effect of allocation, and c) large decision space
- *What is a general solution? Why can we not use it?*
  - *General Solution:* Model performance as a function of SSD allocation and formulate SSD allocation as a global optimization problem.
  - *Baselines:* static partitioning, equal-share allocation, and demand-based heuristic SSD resizing
  - Impractical in practice…
- *What is the suggested solution?*
  - FleetIO proposes a reinforcement-learning–based policy that learns allocation decisions directly from runtime telemetry, enabling adaptive, system-wide SSD allocation without explicit performance modeling.

How Did FleetIO Use RL?

The RL agent operates above the SSD virtualization layer, with admission control and block harvesting acting as safety and enforcement mechanisms.

- *Decentralized RL:* one agent per vSSD for scalable control
- *Implicit Coordination:* agent-aware reward balances local benefit and global contention
- *Coarse Decision-Making:* 2s
- RL model is trained offline and deployed.
- *What is controlled in the hardware?* SSD virtualization can map each virtual SSD (vSSD) to a set of flash channels and chips, and collocate multiple vSSDs on the shared SSD

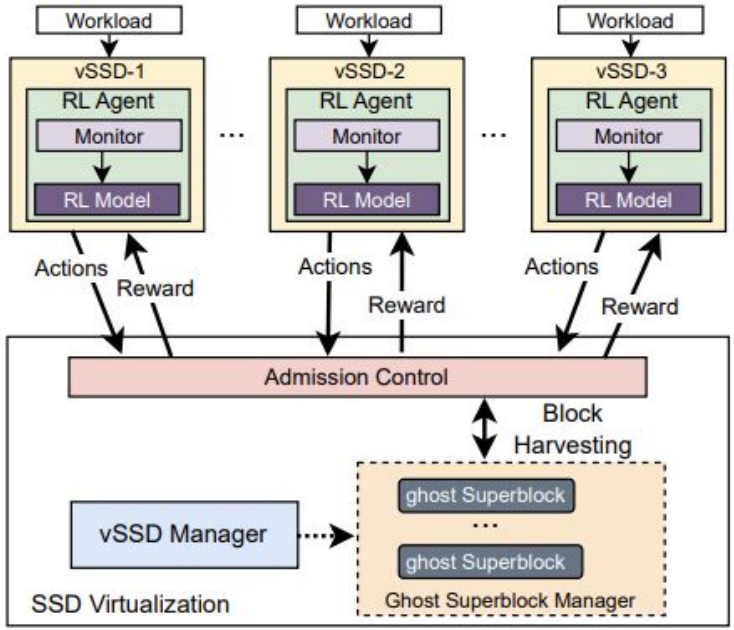


Figure 5. System architecture of FleetIO.

Things The Authors Did Well

The states and actions are well defined. Excellent modelling of the environment and pain points.

Table 1. Definition of RL states in FleetIO.

Symbol	Definition
Avg_BW_t	Average I/O bandwidth of the vSSD (MB/s)
Avg_IOPS_t	Average IOPS of the workload ($reqs/s$)
Avg_Lat_t	Average request latency of the vSSD (us)
SLO_Vio_t	Percentage of vSSD SLO violations (%)
$QDelay_t$	I/O request delay in a vSSD
RW_Ratio_t	Read/write ratio of the workload (%)
$Avail\_Capacity_t$	Available storage capacity (GB)
In_GC_t	Whether the vSSD is performing GC
$Cur\_Priority_t$	The current I/O request priority

Table 2. RL actions defined in FleetIO.

Symbol	Definition
$Harvest(gsb\_bw)$	Storage bandwidth to harvest
$Make\_Harvestable(gsb\_bw)$	Storage bandwidth harvestable for other vSSDs
$Set\_Priority(level)$	Set the I/O request priority

Reward

$$R_{single} = (1-\alpha) * \frac{Avg\_BW_{RL}}{Avg\_BW_{guar}} - \alpha * \frac{SLO\_Vio_{RL}}{SLO\_Vio_{guar}} \quad (1)$$

$$R_{i,\ multi\_agent} = \beta * R_{i,\ single} + (1-\beta) * \frac{\sum_{v \in V}^{v \neq i} R_{v,\ single}}{|V| - 1} \quad (2)$$

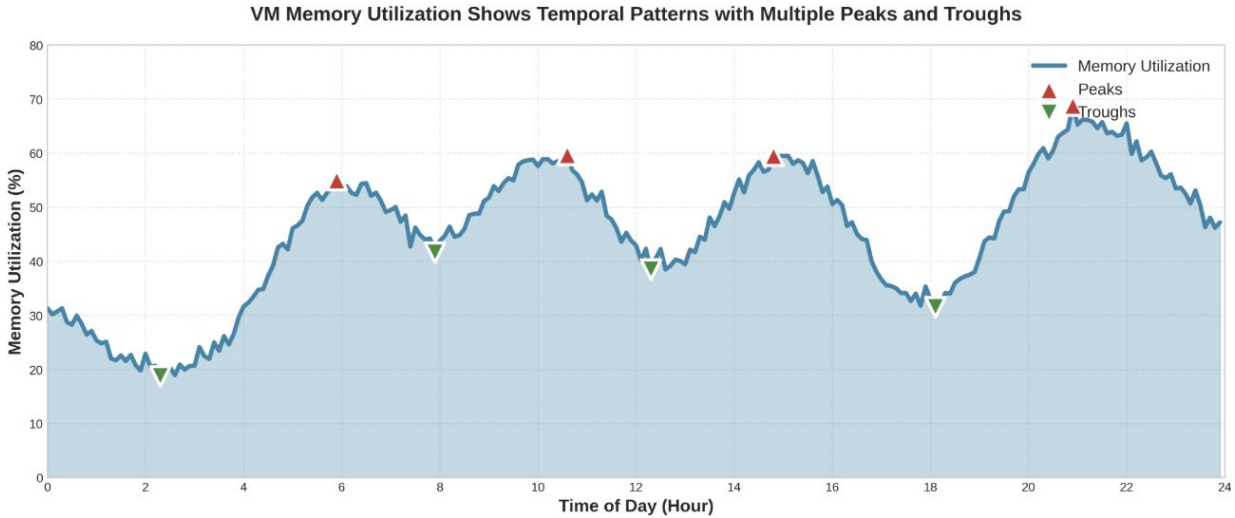
Objective

$$\max_{\pi} \mathbb{E}\left[\sum_{t=0}^{\infty} \lambda^t r_t\right]$$

RL as a Controller

We should use the data's patterns to our advantage!

- Coach (ASPLOS '25) has shown that memory utilization follows predictable daily patterns we can leverage for RL-based allocation.
  - Memory demand isn't random—VMs exhibit *temporal patterns*



Example of RL Policy

The RL Formulation (MDP)

State (st):

Current MHD utilization
Host demand history
Time-of-day features
Pod topology

Action (at):

Allocation vector:
$[x_1, x_2, ..., x_k]$
where $\Sigma x_i = X$ GB
(only from connected MHDs)

Reward (rt):

```
r = β × (util_balance)
    + (1−β) ×
(migration_penalty)

Trades off NOW vs FUTURE
```